

4.1 IF Statement

4.1.1 Simple if Statement

An **if** statement consists of a Boolean expression followed by one or more statements.

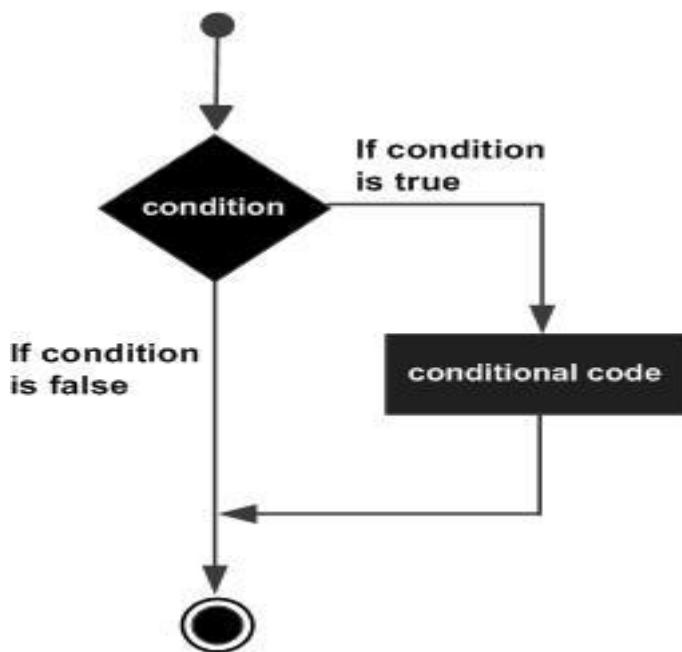
The syntax of an 'if' statement in C programming language is –

```
if(boolean_expression) {  
    /* statement(s) will execute if the boolean expression is true */  
}
```

If the Boolean expression evaluates to **true**, then the block of code inside the 'if' statement will be executed. If the Boolean expression evaluates to **false**, then the first set of code after the end of the 'if' statement (after the closing curly brace) will be executed.

C programming language assumes any **non-zero** and **non-null** values as **true** and if it is either **zero** or **null**, then it is assumed as **false** value.

Flow Diagram of if



Example

```
void main () {  
    int a = 10;  
    if( a < 20 ) {  
        printf("a is less than 20\n" );  
    }  
    printf("value of a is : %d\n", a);  
}
```

Output :

a is less than 20

value of a is : 10

4.1.2 if...else statement

An **if statement** can be followed by an optional **else statement**, which executes when the Boolean expression is false.

The syntax of an 'if...else' statement in C programming language is –

```
if(boolean_expression) {  
    /* statement(s) will execute if the boolean expression is true */  
}  
Else {  
    /* statement(s) will execute if the boolean expression is false */  
}
```

If the Boolean expression evaluates to **true**, then the block of code inside the 'if' statement will be executed. If the Boolean expression evaluates to **false**, then the block of code inside the 'else' statement will be executed.

Example

```
void main () {  
    int a = 21;  
    if( a < 20 ) {  
        printf("a is less than 20\n" );  
    }  
    Else {  
        printf("a is greater than or equal to 20\n" );  
    }  
    printf("value of a is : %d\n", a);  
}
```

Output :

a is greater than or equal to 20

value of a is : 10

4.1.3 Nested if statement

A nested if statements for used when we want to check for a secondary condition if the first condition executes as true. For this, we can have an if-else statement inside of another if-else statement.

The syntax of an 'if...else' statement in C programming language is –

```
if(boolean_expression1) {  
    /* statement(s) will execute if the boolean expression is true */
```

```
        if(boolean_expression2) {
            /* statement(s) will execute if the boolean expression1 is true */
        }

    }
    Else {
        /* statement(s) will execute if the boolean expression1 is false */
    }
}
```

Example

```
void main () {
    int a = 11;
    if( a < 20 ) {
        if(a>10) {
            printf("a is less than 20 and greater than 10 \n" );
        }
        Else {
            printf("a is less than 10 \n" );
        }
    }
    Else {
        printf("a is greater than or equal to 20\n" );
    }

    printf("value of a is : %d\n", a);
}
```

Output :

a is less than 20 and greater than 10

value of a is : 11

4.6 switch statement

A **switch** statement allows a variable to be tested for equality against a list of values. Each value is called a case, and the variable being switched on is checked for each **switch case**.

Syntax

The syntax for a **switch** statement in C programming language is as follows –

```
switch(expression) {  
  
    case constant-expression :  
        statement(s);  
        break; /* optional */  
  
    case constant-expression :  
        statement(s);  
        break; /* optional */  
  
    /* you can have any number of case statements */  
  
    default : /* Optional */  
        statement(s);  
}  

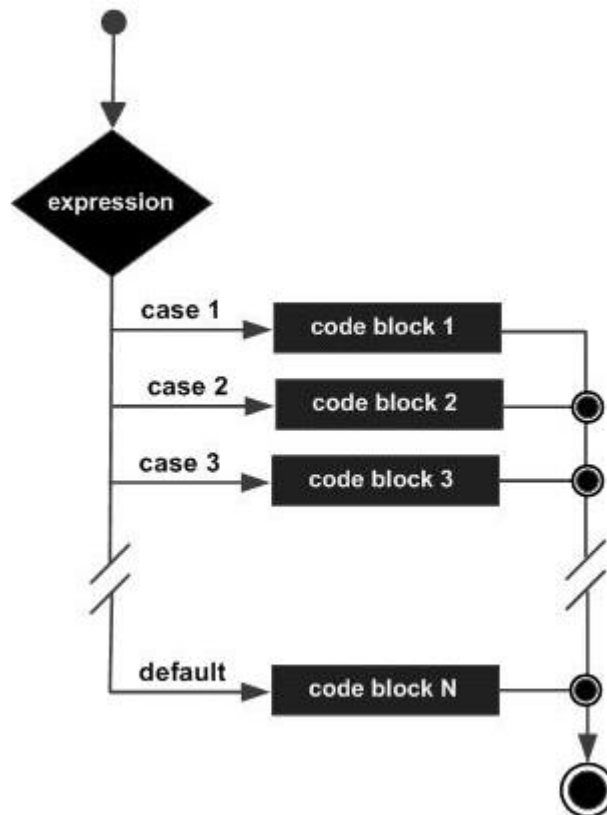
```

The following rules apply to a **switch** statement –

- The **expression** used in a **switch** statement must have an integral or enumerated type, or be of a class type in which the class has a single conversion function to an integral or enumerated type.
- You can have any number of case statements within a switch. Each case is followed by the value to be compared to and a colon.

- The **constant-expression** for a case must be the same data type as the variable in the switch, and it must be a constant or a literal.
- When the variable being switched on is equal to a case, the statements following that case will execute until a **break** statement is reached.
- When a **break** statement is reached, the switch terminates, and the flow of control jumps to the next line following the switch statement.
- Not every case needs to contain a **break**. If no **break** appears, the flow of control will *fall through* to subsequent cases until a break is reached.
- A **switch** statement can have an optional **default** case, which must appear at the end of the switch. The default case can be used for performing a task when none of the cases is true. No **break** is needed in the default case.

Flow Diagram



Example

```
#include <stdio.h>

int main () {
```

```
/* local variable definition */  
char grade = 'B';  
switch(grade) {  
    case 'A' :  
        printf("Excellent!\n" );  
        break;  
    case 'B' :  
    case 'C' :  
        printf("Well done\n" );  
        break;  
    case 'D' :  
        printf("You passed\n" );  
        break;  
    case 'F' :  
        printf("Better try again\n" );  
        break;  
    default :  
        printf("Invalid grade\n" );  
}  
printf("Your grade is %c\n", grade );  
return 0;  
}
```

When the above code is compiled and executed, it produces the following result –

```
Well done  
Your grade is B
```

==> **nested switch statements**

It is possible to have a switch as a part of the statement sequence of an outer switch. Even if the case constants of the inner and outer switch contain common values, no conflicts will arise.

Syntax

The syntax for a **nested switch** statement is as follows –

```
switch(ch1) {  
  
    case 'A':  
        printf("This A is part of outer switch" );  
  
        switch(ch2) {  
            case 'A':  
                printf("This A is part of inner switch" );  
                break;  
            case 'B': /* case code */  
        }  
  
        break;  
    case 'B': /* case code */  
}
```

Example

```
#include <stdio.h>  
  
int main () {  
  
    /* local variable definition */  
  
    int a = 100;  
    int b = 200;  
  
    switch(a) {  
  
        case 100:  
            printf("This is part of outer switch\n", a );  
  
            switch(b) {  
                case 200:  
                    printf("This is part of inner switch\n", a );  
            }  
        }  
  
    printf("Exact value of a is : %d\n", a );  
    printf("Exact value of b is : %d\n", b );
```

```
    return 0;  
}
```

When the above code is compiled and executed, it produces the following result –

```
This is part of outer switch  
This is part of inner switch  
Exact value of a is : 100  
Exact value of b is : 200
```